Practice for the PLF: A Vehicle Rental

> Class: 5AKIF
> Training for the PLF (practical test) on Wednesday, 4 November 2026
> Working time in the PLF: 3 UE (lessons)

Notes on this exercise

This exercise shows you what the task description and the test program look like in the PLF. The PLF has the same structure: a class model, a description of the classes and a test program that awards points.

Download the file *CarRentalService.7z* from the exercises of the chapter *Types*. Extract the file and open the file *CarRentalService.sln*.

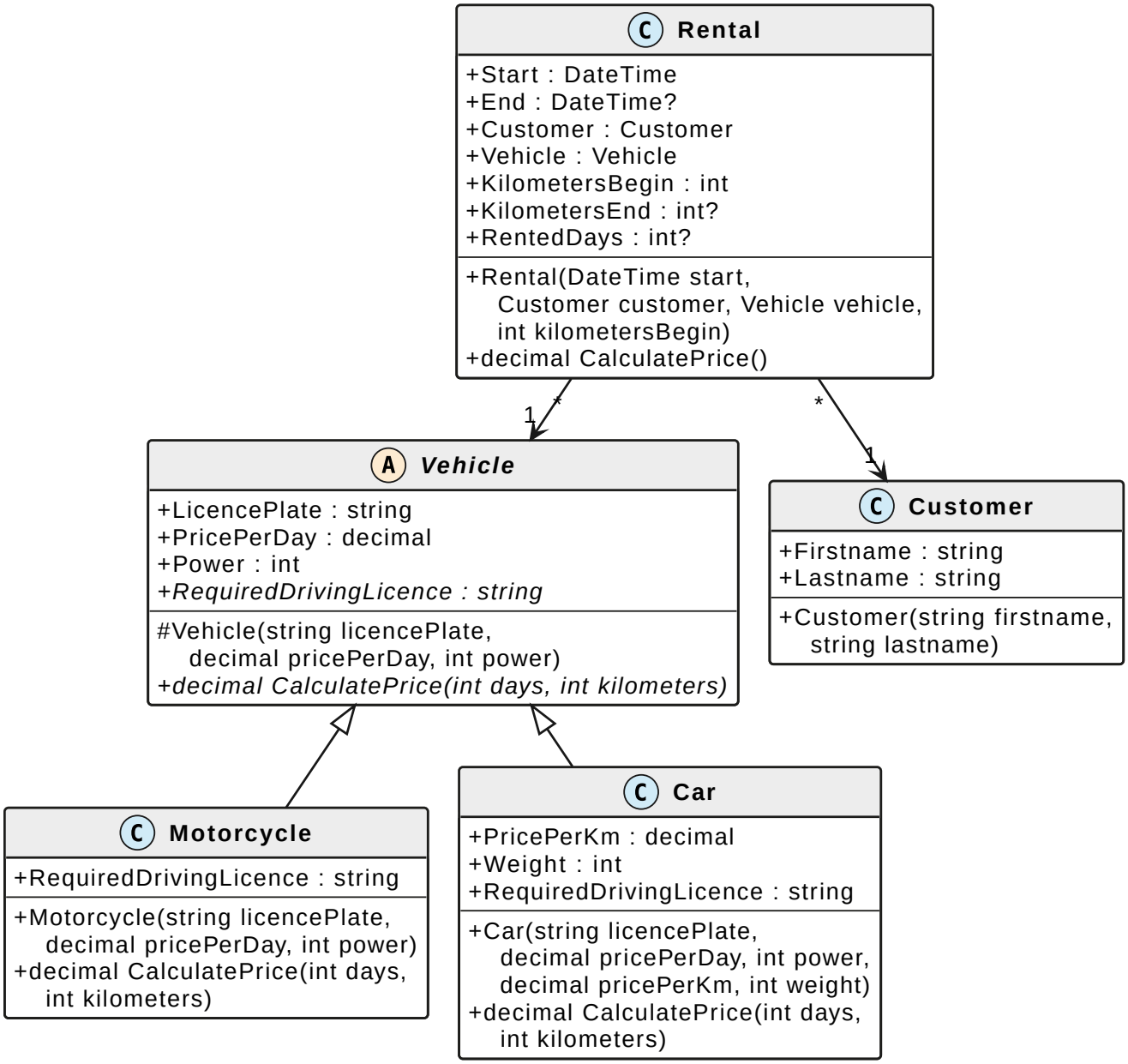


In the PLF you will find the solution on drive *Z*. There you first enter your name in the file *README.md*. In this exercise, however, the file *README.md* is the task description, and you do not have to enter anything.

Task: A vehicle rental

A rental company rents out motorcycles and cars. For a motorcycle you pay a price per day. For a car you also pay for every kilometer driven. For every rental the system stores a rental record (*Rental*). A rental is open until the vehicle is returned.

Class model



Class description

Create the private fields that you need for the implementation yourself.

Class Vehicle

The abstract base class for cars and motorcycles. All properties are read-only.

- **LicencePlate** is the licence plate (number plate) of the vehicle.
- **PricePerDay** is the rental price for every day started.
- **Power** is the power of the vehicle in kW.
- The constructor is *protected*.
- **RequiredDrivingLicence** is an abstract property. It returns the driving licence category that

you need for the vehicle.

- **CalculatePrice()** is an abstract method. It calculates the price from the rental days (*days*) and the kilometers driven (*kilometers*).

Class Motorcycle

All properties are read-only.

- **RequiredDrivingLicence** always returns *"A"*.
- **CalculatePrice()** calculates the price with the formula *days* × *PricePerDay*. The kilometers do not matter.

Class Car

All properties are read-only.

- **PricePerKm** is the price per kilometer driven.
- **Weight** is the weight in kg.
- **RequiredDrivingLicence** returns *"B"* if the car weighs less than 3500 kg. Otherwise it returns *"C"* (truck).
- **CalculatePrice()** calculates the price with the formula *days* × *PricePerDay* + *kilometers* × *PricePerKm*.

Class Customer

The class only stores the first name and the last name of the customer. The constructor sets all properties. All properties are read-only.

Class Rental

The class stores one rental.

- **Start**, **Customer**, **Vehicle** and **KilometersBegin** are set in the constructor and are read-only.
- **Start** is the beginning of the rental.
- **KilometersBegin** is the odometer reading at the beginning of the rental.
- **End** is the end of the rental. It is *null* as long as the vehicle has not been returned. It is set from outside the class when the vehicle is returned.
- **KilometersEnd** is the odometer reading at the end of the rental. It is *null* as long as the vehicle has not been returned. It is set from outside the class when the vehicle is returned.
- **RentedDays** is the number of days started and is read-only. If *End* is *null*, *RentedDays* is also *null*. A rental that ends on the same day lasts 1 day.
- **CalculatePrice()** calculates the price of the rental:
  - If *End* is *null*, the method returns 0.
  - If *KilometersEnd* is *null*, the method returns 0.

- Otherwise it returns the result of *CalculatePrice()* of the rented vehicle. Pass *RentedDays* and the kilometers driven (*KilometersEnd* − *KilometersBegin*) as arguments.

Example: A car with a *PricePerDay* of 10 euros and a *PricePerKm* of 5 euros is rented from 10 November 2021 to 15 November 2021. The odometer reading is 100 km at the beginning and 200 km at the end.

- *RentedDays* is 6 (10, 11, 12, 13, 14 and 15 November).
- *CalculatePrice()* returns 6 × 10 + 100 × 5 = 560 euros.



- If you subtract 2 *DateTime* values, you get a *TimeSpan*. You get the days started with this expression:

```
(int)(end - start).TotalDays + 1
```

- This only works with *DateTime*, not with *DateTime?*. So first check whether *End* is *null*. After that you get the value with `End.Value`. The same applies to *KilometersEnd*.
- You write a *decimal* literal with *M* at the end, e.g. `0.5M`.

Tasks

Create every class in its own file. You must **not** change the files *Program.cs* and *TestHelpers.cs*.

Program.cs is a test program. It only compiles when all classes with all properties and methods from the model exist. Also pay attention to the names of the constructor parameters, because the test program uses them (e.g. `pricePerDay: 10`). So create all classes first. Methods and properties that you have not implemented yet throw a *NotImplementedException*. After that, start the program in the project *CarRentalService.Application*. It shows *OK* or *FAILED* for every check. If a check throws an exception, the check fails. An exception outside of a check (e.g. when the test data is created) ends the program.

A complete solution shows *OK* for all 43 checks. At the end the program shows the points reached and the grade:

```
Result of the program in C:\...\CarRentalService.Application:
43 of 43 checks passed.
48 of 48 points. Grade: Sehr gut.
```

Grading

To get a positive grade, the program must compile. Programs that do not compile or that crash get the grade *Nicht genügend* (fail). The table shows all checks of the test program. The texts are the texts of the screen output. Every check with *OK* gives the points in the column *P*.

No.	Check (48 points in total)	P
Checking class implementation		
1	Rental does not have a default constructor.	1
2	Rental.Start is read-only.	1
3	Rental.Customer is read-only.	1
4	Rental.Vehicle is read-only.	1
5	Rental.KilometersBegin is read-only.	1
6	Rental.RentedDays is read-only.	1
7	Rental.End, KilometersEnd and RentedDays are nullable.	1
8	Customer does not have a default constructor.	1
9	Customer is immutable.	1
10	Vehicle does not have a default constructor.	1
11	Vehicle is abstract.	1
12	Vehicle has a protected constructor.	1
13	Vehicle is immutable.	1
14	Vehicle.PricePerDay is decimal and Vehicle.Power is int.	1
15	Vehicle.RequiredDrivingLicence is abstract.	1
16	Vehicle.CalculatePrice() is abstract.	1
17	Motorcycle does not have a default constructor.	1
18	Motorcycle is immutable.	1
19	Car does not have a default constructor.	1
20	Car is immutable.	1
21	Car.PricePerKm is decimal and Car.Weight is int.	1
Checking constructors		
22	Constructor of Customer sets the values.	1
23	Constructor of Motorcycle sets the values.	1
24	Constructor of Car sets the values.	1
25	Constructor of Rental sets the values.	1
Checking Motorcycle		
26	The required driving licence for motorcycles is A.	1
27	Motorcycle.CalculatePrice() returns days x PricePerDay.	2
Checking Car		
28	The required driving licence for cars < 3500 kg is B.	1
29	Car.CalculatePrice() returns days x PricePerDay + km x PricePerKm.	2

No.	Check (48 points in total)	P
30	The required driving licence for cars >= 3500 kg is C.	1
Checking Rental		
31	The default value of Rental.KilometersEnd is null.	1
32	The default value of Rental.End is null.	1
33	Without an end date Rental.RentedDays is null.	1
34	Without End and KilometersEnd Rental.CalculatePrice() returns 0.	1
35	Rental.RentedDays returns the number of days started.	2
36	Without KilometersEnd Rental.CalculatePrice() returns 0.	1
37	Rental.CalculatePrice() uses the driven kilometers.	2
38	Without End Rental.CalculatePrice() returns 0.	1
39	Rental.RentedDays is 1 if the vehicle is returned on the same day.	1
40	Rental.RentedDays works across the end of a month.	1
41	Rental.RentedDays works across the end of a year.	1
42	Rental.CalculatePrice() charges the days if no kilometers were driven.	1
43	Rental.CalculatePrice() uses the price calculation of the rented vehicle.	2

48 - 42 points: Sehr gut (1, excellent), 41 - 36 points: Gut (2, good), 35 - 30 points: Befriedigend (3, satisfactory), 29 - 24 points: Genügend (4, sufficient), 23 - 0 points: Nicht genügend (5, fail). The program shows the name of the grade in German.